Python Engineering - enumerate() Guide

Qué hace, cuándo usarlo, start=0/1/2 y por qué es mejor que llevar un contador manual

1. ¿Qué es enumerate()?

`enumerate()` es una función incorporada de Python que permite recorrer un iterable y obtener **dos cosas a la vez**: un contador y el elemento actual.

```
items = ['A', 'B', 'C']

for index, item in enumerate(items):
    print(index, item)

0 A
1 B
2 C
```

La idea mental es:

```
enumerate(iterable)
      |
      v
(counter, element)
(counter, element)
(counter, element)
...
```

Por eso solemos escribir `for index, item in enumerate(...)`: Python va entregando parejas y las desempaqueta en dos variables.

2. ¿Qué argumentos recibe enumerate()?

```
enumerate(iterable, start=0)
```

Argumento	Qué significa
iterable	La colección o secuencia que queremos recorrer: list, tuple, string, archivo, etc.
start	Valor inicial del contador. Es opcional. Por defecto vale 0.

Muy importante: **start solo cambia el número del contador**. No hace que Python se salte elementos del iterable.

3. start=0: el comportamiento por defecto

```
letters = ['A', 'B', 'C']

for index, letter in enumerate(letters, start=0):
    print(index, letter)

0 A
1 B
2 C
```

Es exactamente equivalente a escribir simplemente:

```
for index, letter in enumerate(letters):
    ...
```

Usamos start=0 cuando queremos indices tipo Python o cuando el contador representa posiciones que comienzan en cero.

4. start=1: numeración humana

```
names = ['Ana', 'Luis', 'Marta']

for number, name in enumerate(names, start=1):
    print(number, name)

1 Ana
2 Luis
```

```
3 Marta
```

Esto es útil para listas numeradas para personas, informes, menús o mensajes del tipo 'elemento 1, elemento 2...'.

5. ¿Por qué usamos start=2 en EX23?

En el CSV de EX23, la primera línea física del archivo es la cabecera:

```
line 1 -> h_m,v_m_s,t_s
line 2 -> 100.0,20.0,0.0
line 3 -> 102.0,NOT_A_NUMBER,0.1
line 4 -> 103.9,18.0,0.2
```

Primero hacemos:

```
header = file.readline().strip()
```

Después de leer la cabecera, el iterador del archivo está situado en la primera fila de datos, que físicamente es la **línea 2** del archivo. Por eso usamos:

```
for line_number, line in enumerate(file, start=2):
    ...
```

Así, si falla la conversión en la tercera línea física, el mensaje puede decir exactamente `Line 3`.

6. start=2 NO significa 'empieza desde el segundo elemento'

Este es uno de los puntos que más confunden al principio.

```
items = ['A', 'B', 'C']

for n, item in enumerate(items, start=2):
    print(n, item)
```

```
2 A
3 B
4 C
```

Observa que **A sigue siendo el primer elemento recorrido**. Lo único que ha cambiado es el número que asociamos a ese primer elemento.

7. ¿Qué devuelve realmente enumerate()?

`enumerate()` crea un objeto iterable que produce parejas del tipo:

```
(counter, element)
```

Podemos verlo convirtiéndolo temporalmente a list para estudiar el resultado:

```
values = ['A', 'B', 'C']

result = list(enumerate(values, start=1))
print(result)
```

```
[(1, 'A'), (2, 'B'), (3, 'C')]
```

Cada elemento generado es una tuple de dos elementos. El `for index, value` desempaqueta automáticamente esa tuple.

8. ¿Por qué no usar simplemente un for normal?

Un for normal es perfecto cuando solo necesitamos el elemento:

```
for line in file:
    print(line)
```

Pero si además queremos saber **qué número de línea estamos procesando**, necesitamos alguna forma de llevar un contador.

```
for line_number, line in enumerate(file, start=2):
    print(line_number, line)
```

Ese es precisamente el problema que resuelve enumerate().

9. ¿Por qué no usar un contador manual?

Podríamos hacer esto:

```
line_number = 2

for line in file:
    print(line_number, line)
    line_number = line_number + 1
```

Funciona. Pero ahora tenemos que gestionar nosotros mismos el contador.

Con enumerate():

```
for line_number, line in enumerate(file, start=2):
    print(line_number, line)
```

Tenemos menos código, menos posibilidades de olvidar incrementar el contador y una intención más clara: 'recorre esto y dame también su número'.

10. ¿Por qué no usar range(len(...))?

Con una lista podríamos escribir:

```
values = ['A', 'B', 'C']

for i in range(len(values)):
    print(i, values[i])
```

Pero para recorrer directamente elementos suele ser más claro:

```
for i, value in enumerate(values):
    print(i, value)
```

Además, `range(len(...))` necesita que el objeto tenga longitud e indexación. Un archivo es un iterable que podemos recorrer línea a línea, pero no necesitamos cargarlo entero ni indexarlo.

11. ¿Y un 'for con dos variables' sin enumerate()?

Es importante entender que escribir dos variables después de `for` no crea automáticamente un contador.

```
for a, b in iterable:
    ...
```

Esto significa: **cada elemento del iterable debe poder desempaquetarse en dos valores**.

```
pairs = [(10, 20), (30, 40)]

for a, b in pairs:
    print(a, b)
```

Aquí funciona porque cada elemento ya es una tuple de dos valores. En un archivo, cada elemento es una línea de texto, no una pareja (número, línea). `enumerate(file)` crea precisamente esas parejas.

12. Comparación de alternativas

Necesidad	Opción recomendada
Solo necesito el elemento	for item in iterable
Necesito elemento + contador	for i, item in enumerate(iterable)
Quiero que el contador empiece en 1	enumerate(iterable, start=1)
Quiero reflejar líneas físicas tras una cabecera	enumerate(file, start=2)
Tengo parejas ya existentes	for a, b in pairs
Necesito acceder por índice de forma explícita	range(len(sequence)) puede tener sentido

13. Ejemplo completo fuera de archivos

```
temperatures = [18.2, 19.5, 21.1]

for measurement_number, temperature in enumerate(
    temperatures,
    start=1
):
    print(
        f'Measurement {measurement_number}: '
        f'{temperature} C'
    )
```

```
Measurement 1: 18.2 C
Measurement 2: 19.5 C
Measurement 3: 21.1 C
```

14. Ejemplo completo aplicado a EX23

```
with open(file_path, 'r', encoding='utf-8') as file:
    header = file.readline().strip()

    for line_number, line in enumerate(file, start=2):
        clean_line = line.strip()
        values = clean_line.split(',')

        if len(values) != 3:
            raise ValueError(
                f'Line {line_number}: expected 3 columns'
            )
```

Aquí `line_number` no sirve para calcular nada físico. Sirve para **diagnóstico y depuración**: saber en qué línea real del archivo apareció el problema.

15. Chuleta final

```
enumerate(iterable)
-> counter starts at 0

enumerate(iterable, start=1)
-> counter starts at 1

enumerate(iterable, start=2)
-> counter starts at 2
-> DOES NOT skip the first two elements

for i, value in enumerate(data):
    ...

USE IT WHEN:
- you need the element
- and also a counter/index/line number
```

Idea central: enumerate() no cambia lo que recorres. Solo añade una numeración útil a cada elemento.